Automation Auditor Final Report

Samuel Lachisa ·

Feb 28 2026

Executive Summary

The Automaton Auditor is a LangGraph-based “Digital Courtroom” that autonomously audits a GitHub repository and its architectural PDF report. The system implements three conceptual layers: **Detectives** (evidence acquisition via RepoInvestigator, DocAnalyst, VisionInspector), **Judges** (dialectical evaluation by Prosecutor, Defense, and Tech Lead personas), and **Chief Justice** (deterministic synthesis using hardcoded rules such as security override, fact supremacy, and dissent summarization). State is strongly typed with Pydantic models and reducer-annotated AgentState (operator.ior for evidences, operator.add for opinions) to support parallel execution. The StateGraph in src/graph.py implements two fan-out/fan-in patterns (Detectives → EvidenceAggregator, Judges → ChiefJustice) and conditional routing for evidence-missing scenarios. This report documents the architecture rationale, a deep dive into Dialectical Synthesis, Fan-In/Fan-Out, and Metacognition with concrete code references, gap analysis and forward plan, StateGraph diagram, criterion-by-criterion self-audit template, MinMax feedback reflection, and a consolidated remediation plan.

Part 1. Architecture Decision Rationale

1.1 Overall Architectural Intent

The Automaton Auditor is designed as a LangGraph-based "Digital Courtroom" that autonomously audits a GitHub repository and its architectural PDF report. The system explicitly operationalizes three conceptual layers:

- **Layer 1 – Detectives (Evidence Acquisition)**
  RepoInvestigator, DocAnalyst, and VisionInspector operate over code, PDF documentation, and optional diagrams. Their shared output is standardized as Pydantic Evidence objects and accumulated in the typed AgentState.

- **Layer 2 – Judges (Dialectical Evaluation)**
  Three distinct judge personas (Prosecutor, Defense, Tech Lead) consume rubric-aware evidence and produce structured JudicialOpinion objects, each focused on a rubric dimension (e.g., git_forensic_analysis, graph_orchestration). This layer implements dialectical synthesis inputs: conflicting viewpoints over the same evidence.

- **Layer 3 – Chief Justice (Deterministic Synthesis)**
  A Chief Justice node aggregates all opinions, applies deterministic synthesis rules (e.g., security override, fact supremacy, functionality weighting), and produces a final AuditReport rendered as Markdown.

This layered design is explicitly aligned with the rubric's expectations around fan-out/fan-in patterns, dialectical synthesis, and state synchronization.

1.2 State Management and Typed Models

The project adopts a strongly typed, reducer-based state model in src/state.py:

- **Evidence Representation** – Evidence (BaseModel) captures: goal (forensic question being answered), found (existence), content (snippet), location (file path / commit hash), rationale (why this evidence is trustworthy), confidence (0–1).

- **Judicial Opinions** – JudicialOpinion (BaseModel) encodes: judge ("Prosecutor" | "Defense" | "TechLead"), criterion_id (rubric dimension id), score (1–5), argument (reasoning), cited_evidence (IDs / snippets).

- **Synthesis Outputs** – CriterionResult and AuditReport (BaseModel) capture: per-dimension final ruling plus dissent and remediation, and overall audit metadata and remediation plan.

- **Graph State** – AgentState is a TypedDict with annotated reducers:

  - evidences: Annotated[Dict[str, List[Evidence]], operator.ior]
  - opinions: Annotated[List[JudicialOpinion], operator.add]

Using operator.ior and operator.add as reducers is an intentional decision to guarantee parallel-safe state synchronization across Detectives and Judges:

- evidences merges dictionaries instead of overwriting, so multiple Detectives can add evidence under the same rubric dimension concurrently.
- opinions concatenates lists, allowing Judges to emit multiple JudicialOpinion instances safely.

This design directly addresses the rubric's State Management Rigor success pattern and avoids the failure case of plain dict overwrites.

1.3 Graph Orchestration via LangGraph StateGraph

The orchestration is centered in src/graph.py, using StateGraph(AgentState):

- **Detective Fan-Out / Fan-In**
  - Nodes: RepoInvestigator, DocAnalyst, VisionInspector, EvidenceAggregator.
  - Edges: START → RepoInvestigator, START → DocAnalyst, START → VisionInspector; each Detective → EvidenceAggregator.
  - This realizes parallel fan-out from START to three Detectives and fan-in at EvidenceAggregator.
- **Evidence-Sensitive Conditional Routing**
  - After aggregation, the graph computes _evidence_status(state: AgentState) -> "ok" | "missing", inspecting state["evidences"] for critical dimensions: git_forensic_analysis, state_management_rigor, graph_orchestration, safe_tool_engineering.
  - builder.add_conditional_edges("EvidenceAggregator", _evidence_status, {"ok": "JudgesEntry", "missing": "EvidenceMissing"}) ensures: if required evidence is missing, the flow routes to EvidenceMissing; otherwise execution continues to JudgesEntry, which fans out to Judges.
- **Judge Fan-Out / Fan-In**
  - Nodes: Prosecutor, Defense, TechLead, JudgesEntry, ChiefJustice.
  - Edges: JudgesEntry → Prosecutor, JudgesEntry → Defense, JudgesEntry → TechLead; each Judge plus EvidenceMissing → ChiefJustice; ChiefJustice → END.

This structure implements a second fan-out/fan-in pattern for Judges, aligned with the rubric: Detectives and Judges both run in parallel, with explicit synchronization points and error-aware branching.

1.4 LLM Tooling and Structured Output

Within src/nodes/judges.py, Judge behavior is defined as follows:

- **Distinct Personas (Judicial Nuance)**

  - PROSECUTOR_SYSTEM: adversarial, security-focused, rewards finding gaps, flags hallucinations (e.g., "Hallucination Liability").
  - DEFENSE_SYSTEM: effort / intent-oriented, emphasizes creativity and struggle.
  - TECH_LEAD_SYSTEM: pragmatic, cares about reducers, sandboxing, and maintainability.

  Each persona is deliberately worded to diverge in philosophy, satisfying the rubric's requirement for dialectical tension, not mere prompt copies.

- **Structured Output Enforcement**
  - _invoke_judge calls llm.with_structured_output(JudicialOpinion), then normalizes: accepts either JudicialOpinion or a dict convertible into it; raises an error if neither, and the caller wraps exceptions into a low-score JudicialOpinion.
  - _judge_node iterates over rubric_dimensions in state, invoking each Judge across all dimensions and appending JudicialOpinion instances to opinions.

This approach gives schema-first enforcement of structured outputs, aligned with the rubric's Structured Output Enforcement dimension.

1.5 Chief Justice and Synthesis Rules

src/nodes/justice.py implements deterministic rules (hardcoded Python logic, not LLM prompts):

- **Security Override** – Confirmed security flaws cap scores (e.g., at 3).
- **Fact Supremacy** – Forensic evidence (Detectives' facts) overrules Judicial opinion (Judges' narratives). If Defense claims "Deep Metacognition" but Detective evidence shows found=False, Defense is overruled.
- **Functionality Weight** – Tech Lead's architectural confirmation carries elevated influence for architecture-related dimensions.
- **Dissent Requirement / Variance Re-evaluation** – When score variance across Judges exceeds 2, the Chief Justice uses Tech Lead as tie-breaker and writes a mandatory dissent_summary into CriterionResult.

These rules are encoded in rubric/week2_rubric.json and implemented as deterministic Python if/else logic in _resolve_score(), chief_justice_node(), and related helpers.

Part 2. Architecture Deep Dive: Concepts with Substance

This section grounds each rubric concept in concrete code, not buzzwords.

2.1 Dialectical Synthesis

Definition: Dialectical synthesis is the process of reconciling opposing viewpoints (thesis vs. antithesis) into a reasoned conclusion. In the auditor, three Judge personas embody this:

- **Prosecutor** (thesis: harsh, gap-finding) – PROSECUTOR_SYSTEM in src/nodes/judges.py instructs: "Trust No One. Assume Vibe Coding." Argues for low scores on missing evidence.
- **Defense** (antithesis: generous, intent-focused) – DEFENSE_SYSTEM instructs: "Reward Effort and Intent. Look for the Spirit of the Law."
- **Tech Lead** (synthesis: pragmatic tie-breaker) – TECH_LEAD_SYSTEM instructs: "Does it actually work? Is it maintainable?"

Implementation: All three Judges run in parallel (fan-out from JudgesEntry in src/graph.py) on the same evidences per rubric dimension. The Chief Justice in src/nodes/justice.py applies _resolve_score() to combine their opinions: when variance > 2, Tech Lead's score wins; otherwise a weighted average (p + d + 2*t) / 4 is used.

2.2 Fan-In / Fan-Out

Definition: Fan-out = one node sending to multiple successors; fan-in = multiple nodes converging on one successor.

Implementation in src/graph.py:

1. **Detective Fan-Out:** builder.add_edge(START, "RepoInvestigator"), add_edge(START, "DocAnalyst"), add_edge(START, "VisionInspector") – three parallel branches from START.
2. **Detective Fan-In:** add_edge("RepoInvestigator", "EvidenceAggregator"), same for DocAnalyst and VisionInspector – all three merge at EvidenceAggregator.
3. **Judge Fan-Out:** add_edge("JudgesEntry", "Prosecutor"), same for Defense and TechLead.
4. **Judge Fan-In:** Prosecutor, Defense, TechLead, and EvidenceMissing all add_edge(..., "ChiefJustice").

2.3 Metacognition

Definition: Metacognition is the system's ability to reason about its own reasoning quality. In the auditor, this appears as:

- **Confidence in evidence** – Each Evidence has a confidence (0–1) field; Detectives set it based on how strong the forensic finding is.

- **Dissent summarization** – When Judges disagree (variance > 2), the Chief Justice must produce a dissent_summary in CriterionResult – the system reflects on *why* the judges diverged.
- **Evidence-missing routing** – _evidence_status() inspects evidences and routes to EvidenceMissing when critical dimensions lack valid evidence; the graph "knows" when its inputs are insufficient.
- **Fact supremacy** – The Chief Justice overrules Defense opinions when Detective evidence shows found=False; the system prioritizes factual evidence over interpretive narratives.

Part 3. Gap Analysis and Forward Plan

This section evaluates the implemented architecture against each rubric dimension and proposes targeted improvements.

3.1 Git Forensic Analysis

- **Current Strengths:** Repository structure is clean. The forensic tools in src/tools/repo_tools.py (wrapping git_tools.py) implement sandboxed git interactions: temporary clone directories, subprocess.run with timeouts, and explicit error mapping.

- **Gaps / Risks:** git_forensic_analysis evidence does not yet enumerate: total commit count, temporal progression (setup → tool engineering → graph orchestration), or detection of bulk uploads vs. iterative development. There is no explicit README subsection describing how RepoInvestigator surfaces failed forensic checks (e.g., single init commits).

- **Forward Plan:** Enhance RepoInvestigator to attach explicit Evidence items for total commit count, time-series of commits, and presence/absence of bulk uploads under git_forensic_analysis. Document forensics in the architecture report.

3.2 State Management Rigor

- **Current Strengths:** Pydantic models Evidence, JudicialOpinion, CriterionResult, AuditReport are well-typed. AgentState uses TypedDict plus Annotated reducers (operator.ior, operator.add). The evidences structure is keyed by rubric dimension id.

- **Gaps / Risks:** No explicit evidence IDs separate from dimension IDs; cited_evidence is a list of strings (snippets). No provenance metadata (which Detective supplied which evidence).

- **Forward Plan:** Extend Evidence with evidence_id: str and source_agent: Literal["RepoInvestigator", "DocAnalyst", "VisionInspector"]. Encourage Judges to cite by ID for better traceability.

3.3 Graph Orchestration Architecture

- **Current Strengths:** StateGraph clearly encodes parallel Detectives with EvidenceAggregator, conditional routing via _evidence_status, parallel Judges via JudgesEntry, and ChiefJustice fan-in. Conditional edges for evidence-missing implement a meaningful error path.

- **Gaps / Risks:** Only one failure mode ("missing" vs "ok"); no richer states (incomplete, low_confidence, error). No generalized retry/fallback for failed tools (e.g., PDF parsing fails but repo analysis succeeds).

- **Forward Plan:** Extend _evidence_status to return richer states and add corresponding nodes (e.g., remediation node for incomplete evidence). Introduce optional second-pass Detective runs when Judges detect missing evidence.

3.4 Safe Tool Engineering

- **Current Strengths:** src/tools/ is the canonical location for sandboxed git, AST, and PDF tools. The rubric favors tempfile.TemporaryDirectory + subprocess.run; this project uses that pattern in git_tools.py (cloned into tempfile.mkdtemp, subprocess.run with list args, no os.system).

- **Gaps / Risks:** The report should verify: git cloning always in a temporary directory; no direct os.system calls; authentication errors surfaced as evidence.

- **Forward Plan:** Audit src/tools/git_tools.py (via repo_tools.py) to confirm tempfile.mkdtemp around clones, subprocess.run with error handling, and mapping of git failures into Evidence. Add a "Security and Safety" subsection to the architecture report (see Part 7).

3.5 Structured Output Enforcement

- **Current Strengths:** Judges bind to JudicialOpinion via with_structured_output(JudicialOpinion). Fallback logic coerces dict-like outputs into the schema. Errors produce explicit low-score opinions with explanatory arguments.

- **Gaps / Risks:** No retry on malformed outputs before falling back to error opinions. No post-processing validation that score is in [1–5] and argument is non-trivial.

- **Forward Plan:** Introduce a retry wrapper around _invoke_judge: on validation failure, add clarifying system message and retry once before emitting error opinion. Add defensive score normalization and argument validation.

3.6 Judicial Nuance and Dialectics

- **Current Strengths:** Prosecutor, Defense, and Tech Lead prompts are substantively distinct with different priorities (security vs. effort vs. practicality). Judges run in parallel on the same Evidence, ensuring genuine dialectical tension.

- **Gaps / Risks:** No quantification of score divergence per dimension; no explicit check for persona "collusion" (all three returning identical scores/arguments).
- **Forward Plan:** Add Chief Justice logic to detect when all three Judges are effectively identical and flag as potential persona collapse. Surface a "Dissent and Convergence" section in the Markdown report.

3.7 Chief Justice Synthesis Engine

- **Current Strengths:** src/nodes/justice.py implements deterministic Python logic: security override, fact supremacy, Tech Lead weight, variance-based tie-breaker (when variance > 2), and dissent summarization. The final Markdown report contains Executive Summary, Criterion Breakdown (with dissent), and Remediation Plan.
- **Gaps / Risks:** Variance threshold (2) and dissent summarization logic should be audited to ensure they match rubric expectations for every high-variance dimension.
- **Forward Plan:** Ensure ChiefJustice computes per-dimension variance, writes mandatory dissent_summary for variance > 2, and that the report structure is fixed (Executive Summary, Criterion Breakdown with dissent, Remediation Plan, optional LangSmith trace link).

3.8 Theoretical Depth, Report Accuracy, and Diagrams

- **Current Strengths:** The architecture naturally embodies Fan-In/Fan-Out, Dialectical Synthesis, and State Synchronization. This report defines each concept and ties it to concrete code. File paths referenced in the report (e.g., src/state.py, src/graph.py, src/nodes/judges.py, src/nodes/justice.py, src/tools/repo_tools.py, src/tools/doc_tools.py) exist in the repository.
- **Gaps / Risks:** Ensure every referenced path in the PDF is verified; DocAnalyst and Chief Justice cross-reference mentioned vs. actual paths for report_accuracy.
- **Forward Plan:** Continue explicit definition of concepts with code references. Use doc_tools.extract_paths_from_text and cross_reference_paths to verify report paths against repo evidence.

Part 4. StateGraph Architecture – Diagram and Explanation

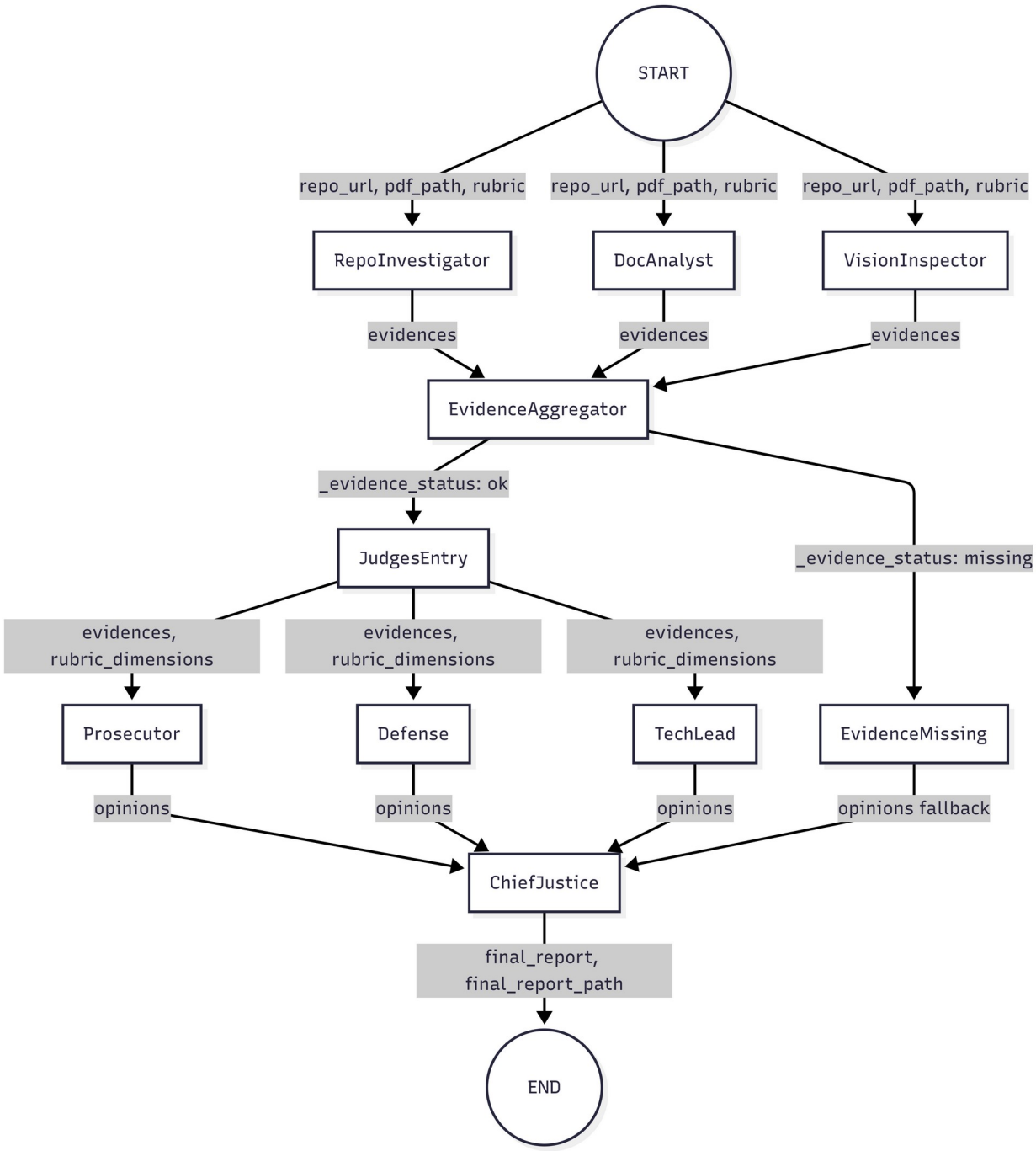
4.1 Textual StateGraph Description

The core LangGraph StateGraph in src/graph.py can be described as:

- **Nodes:** START, END; RepoInvestigator, DocAnalyst, VisionInspector, EvidenceAggregator; JudgesEntry, EvidenceMissing; Prosecutor, Defense, TechLead; ChiefJustice.
- **Edges and Control Flow:**
  - **Detective Fan-Out / Fan-In:** START fans out to RepoInvestigator, DocAnalyst, VisionInspector. All three feed into EvidenceAggregator.
  - **Conditional Routing:** _evidence_status(state) inspects state["evidences"] for critical dimensions. If any critical dimension has evidence where found is consistently False, returns "missing"; otherwise "ok". add_conditional_edges("EvidenceAggregator", _evidence_status, {"ok": "JudgesEntry", "missing": "EvidenceMissing"}) implements the Evidence Missing branch.
  - **Judge Fan-Out / Fan-In:** JudgesEntry fans out to Prosecutor, Defense, TechLead. Prosecutor, Defense, TechLead, and EvidenceMissing all funnel into ChiefJustice. ChiefJustice connects to END.

This structure makes error-aware routing, parallel evaluation, and synchronization points explicit.

4.2 StateGraph Diagram



The diagram makes explicit: Detective fan-out/fan-in; evidence-aware routing (ok → JudgesEntry, missing → EvidenceMissing); Judge fan-out/fan-in; ChiefJustice producing the final AuditReport at END.

Part 5. Criterion-by-Criterion Self-Audit Results

Run the auditor on your own repo and paste results here, or use the template below.

Criterion	Score (1–5)	Prosecutor	Defense	Tech Lead	Notes
git_forensic_analysis	—	—	—	—	Commit count, progression story
state_management_rigor	—	—	—	—	Reducers, Pydantic
graph_orchestration	—	—	—	—	Fan-out/fan-in, conditional edges
safe_tool_engineering	—	—	—	—	No os.system, tempfile
structured_output_enforcement	—	—	—	—	with_structured_output(JudicialOpinion)
judicial_nuance	—	—	—	—	Distinct personas
chief_justice_synthesis	—	—	—	—	Hardcoded rules
theoretical_depth	—	—	—	—	PDF keywor

Criterion	Score (1–5)	Prosecutor	Defense	Tech Lead	Notes
					ds
report_accuracy	—	—	—	—	Verified vs hallucinated paths
swarm_visual	—	—	—	—	Diagram alignment

To populate: uv run run-audit --repo-url . --pdf-path audit/pdf_cache/architectural_report.pdf --output-dir audit/report_onself_generated

Part 6. Reflection on the MinMax Feedback Loop

6.1 What Your Peer's Agent Caught That You Missed

[Fill in after peer audit. Example: "The peer agent flagged that our safe_tool_engineering check did not verify git authentication error handling. We had assumed subprocess.run error mapping was sufficient, but the rubric explicitly asks for authentication failure handling."]

6.2 How You Updated Your Agent to Detect Similar Issues in Others

[Fill in after you extend your agent. Example: "We added a check in RepoInvestigator for explicit error handling around git clone stderr, and we extended the safe_tool_engineering Evidence rationale to cite the presence or absence of auth-error mapping."]

Part 7. Remediation Plan for Remaining Gaps

Gap	Location	Remediation
Evidence IDs and provenance	src/state.py	Add evidence_id: str and source_agent: Literal["RepoInvestigator", "DocAnalyst", "VisionInspector"] to Evidence; have Judges cite by ID.
Richer routing states	src/graph.py	Extend _evidence_status to return "incomplete", "low_confidence", "error"; add nodes for targeted re-collection.
Retry on malformed	src/nodes/judges.py	Wrap _invoke_judge with retry: on validation failure, add clarifying system

Gap	Location	Remediation
Judge output		message and retry once before emitting error opinion.
Persona collapse detection	src/nodes/justice.py	Add logic to flag when all three Judges return identical scores/arguments; surface “Dissent and Convergence” in report.
Git forensic granularity	src/nodes/detectives.py	Attach explicit Evidence for commit count, time-series, bulk-upload detection under git_forensic_analysis.
Security subsection	This report	See below.

7.1 Security and Safety

The auditor enforces a sandboxing pattern for all external operations:

- **Git operations** (src/tools/git_tools.py, exposed via src/tools/repo_tools.py): Clones run inside tempfile.mkdtemp(prefix="auditor_clone_"). subprocess.run() is used with list arguments (no shell). No os.system calls. Clone failures (timeout, git not found, invalid URL) are mapped into Evidence with found=False and explanatory rationale.
- **PDF parsing** (src/tools/doc_tools.py, wrapping pdf_tools.py): File-based extraction via PyMuPDF or Docling; no arbitrary code execution.
- **Vision analysis** (optional, src/tools/vision.py): Sends image bytes to configured vision API; no local code execution on untrusted input.

Part 8. Summary

The Automaton Auditor implements a three-layer “Digital Courtroom” architecture—Detectives, Judges, and Chief Justice—backed by strongly typed state, reducer-based synchronization, and a LangGraph StateGraph that separates parallel evidence collection from dialectical evaluation and deterministic synthesis. Typed models (Evidence, JudicialOpinion, CriterionResult, AuditReport) and structured-output LLM calls ensure judgments are explainable and schema-conformant. The Chief Justice applies explicit synthesis rules (security override, fact supremacy, Tech-Lead weighting, dissent summaries) to produce a rigorous Markdown audit report. Git tooling, AST/PDF analysis, and safe subprocess-based operations align with rubric expectations. Dialectical Synthesis, Fan-In/Fan-Out, and Metacognition are realized in concrete code. The forward plan addresses evidence IDs, richer routing, retry logic, persona collapse detection, deeper git forensics, and a documented Security and Safety subsection.